

Week5 Session 3- Facilitation Guide

(Package Management with NPM)

Index

- I. Recap of the previous session**
- II. What is NPM (Node Package Manager)?**
- III. Installing and Using Packages (npm install)**
- IV. Understanding package.json and package-lock.json**
- V. Managing dependencies (devDependencies vs. dependencies)**

(2hr ILT)

I. Recap of the previous session

In the previous session, we delved into key foundational concepts of Node.js, a versatile JavaScript runtime environment designed to build scalable and high-performance server-side applications. We began by understanding the difference between CommonJS and ES6 modules, which are essential for organizing code in Node.js projects. We explored how `require` and `import` allow us to include and reuse functionality across files, with CommonJS being the default in Node.js and ES6 modules offering a more modern approach.

We also explored built-in modules like `fs`, `path`, and `os`, which provide functionality for file operations, path management, and system-level information, respectively. These modules demonstrated how Node.js offers a rich standard library for handling common tasks.

The session included hands-on practice with the `fs` module, where we learned to read from and write to files, append data, and delete files programmatically. We discussed the importance of asynchronous methods like `fs.readFile` and `fs.writeFile` and how they enhance performance by avoiding blocking operations.

In today's session, we will explore `npm` (Node Package Manager), the backbone of Node.js's vast ecosystem of libraries and tools.

By the end of this session, you will be equipped to efficiently manage packages and dependencies, a crucial skill for real-world Node.js development.

II. What is NPM (Node Package Manager)?

`npm` (Node Package Manager) is the default package manager for Node.js. It is a command-line tool that enables developers to discover, install, share, and manage open-source packages (libraries or modules) for Node.js. These packages provide reusable code for various tasks, helping developers save time and effort during application development.

`npm` is also an online repository where developers can publish their own packages for others to use. It is the largest ecosystem of open-source libraries in the world.

Key Features of npm

- **Install Packages** : With `npm`, you can easily install libraries or tools for your Node.js project using a simple command. For example, installing `express`, a popular web framework, can be done with:

`npm install express`

- **Dependency Management** : npm manages all the dependencies (other packages) your project needs, ensuring that they are stored locally in your project directory and loaded correctly.
- **Versioning**: It allows you to specify the version of the package you want to use. This ensures consistency across projects by avoiding unexpected updates.
- **Global and Local Installation** :
Local packages are installed in your project's directory and are only accessible within the project.
Global packages are installed system-wide and can be used from any project.
- **Custom Scripts** : npm allows you to define and run custom scripts (e.g., build, test, start) specified in the package.json file.
- **Publishing and Sharing** : Developers can publish their own Node.js modules to the npm registry, making them available to the entire Node.js community.

How npm Works

npm uses the package.json file to manage a project's metadata, dependencies, and scripts. When you install a package, npm downloads it from its online registry and stores it in the node_modules folder.

Real-World Applications of npm

- **Web Frameworks** : Installs frameworks like express to build REST APIs or web applications.
- **Build Tools** : Uses tools like webpack or parcel for bundling and optimizing your JavaScript code.
- **Testing** : Installs testing libraries like jest to write and run automated tests.
- **Utilities** : Installs packages like dotenv to manage environment variables securely.

Why npm is Essential

- **Efficient Development** : Reuse existing solutions rather than reinventing the wheel.
- **Collaborative Workflow** : Easily share and manage dependencies across teams.
- **Large Ecosystem** : Access millions of packages to address a variety of programming needs.

By mastering npm, developers can greatly enhance their productivity and streamline the Node.js development process.

III. Installing and Using Packages (npm install)

The npm install command is one of the most powerful features of npm (Node Package Manager). It allows developers to add, install, and manage external libraries (packages) to their Node.js projects, saving time and effort by reusing existing code.

The **npm install** command is used to:

- Install packages locally to your project or globally to your system.
- Install dependencies listed in the package.json file.
- Create a node_modules directory to store installed packages.
- Update the package-lock.json file (if it exists).

Installing Modules using npm

Following is the syntax to install any Node.js module:

npm install <Module Name>

Let's install a famous Node.js web framework called express:

Open the Node.js command prompt and execute the following command:

npm install express

You can see the result after installing the "express" framework.

```
Node.js command prompt x + v

C:\nodejs programs>npm install express

added 69 packages, and audited 71 packages in 6s

14 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

You can use the **npm ls** command to list down all the locally installed modules.

Open the Node.js command prompt and execute :

npm ls

```
Node.js command prompt x + v

C:\nodejs programs>npm ls
nodejs programs@ C:\nodejs programs
+-- express@4.21.2
`-- lodash@4.17.21

C:\nodejs programs>
```

- **Global vs Local Installation**

Local Installation (default) : By default, npm installs dependency in local mode. Here local mode specifies the folder where Node application is present.

Command:

npm install <package-name>

For example if you installed express module, it created node_modules directory in the current directory where it installed express module.

```
Node.js command prompt x + v

C:\nodejs programs>npm ls
nodejs programs@ C:\nodejs programs
+-- express@4.21.2
`-- lodash@4.17.21

C:\nodejs programs>
```

Global Installation : Globally installed packages/dependencies are stored in system directory.

Command:

npm install -g <package-name>

Let's install express module using global installation. Although it will also produce the same result, modules will be installed globally.

```
Node.js command prompt x + v

C:\nodejs programs>npm install -g express

added 69 packages in 1s

14 packages are looking for funding
  run `npm fund` for details

C:\nodejs programs>npm ls -g
C:\Users\Anudip\AppData\Roaming\npm
+-- create-react-app@5.0.1
+-- express@4.21.2
+-- nodemon@3.1.9
`-- yarn@1.22.22

C:\nodejs programs>
```

- **Installing All Project Dependencies**

If a package.json file exists in your project, you can install all its listed dependencies using:
npm install

This reads the dependencies and devDependencies in the package.json file and installs them into the node_modules folder.

- **Using Installed Packages**

Once a package is installed, you can use it in your project by requiring or importing it.

Example:

Install the lodash package:

```
npm install lodash
```

Use it in your project:

```
// Import the lodash module
const loadashmodule= require('lodash');
// Example 1: Chunk an array into smaller arrays
const numbers = [1, 2, 3, 4, 5, 6, 7, 8];
const chunked = loadashmodule.chunk(numbers, 3);
console.log('Chunked Array:', chunked);
```

Output:

```
Node.js command prompt x + v

C:\nodejs programs>node chalkexample.js
Chunked Array: [ [ 1, 2, 3 ], [ 4, 5, 6 ], [ 7, 8 ] ]

C:\nodejs programs>
```

- **Removing Packages**

To remove a package, use:

```
npm uninstall <package-name>
```

Example:

```
npm uninstall lodash
```

- **Updating Packages**

To update a package to its latest version:

```
npm update <package-name>
```

To update all packages:

```
npm update
```

- **Common Commands with npm install**

Commands	Description
npm install	Installs all dependencies from package.json
npm install <package-name>	Installs a package locally.

<code>npm install -g <package-name></code>	Installs a package globally.
<code>npm install <package-name> --save-dev</code>	Installs a package as a dev dependency.
<code>npm uninstall <package-name></code>	Removes a package.
<code>npm install --force</code>	Forcibly installs a package, ignoring errors
<code>npm install --legacy-peer-deps</code>	Ignores incompatible peer dependency warnings

- **Example: Real-World Usage**

Scenario: Building a Simple Server

Install express:

```
npm install express
```

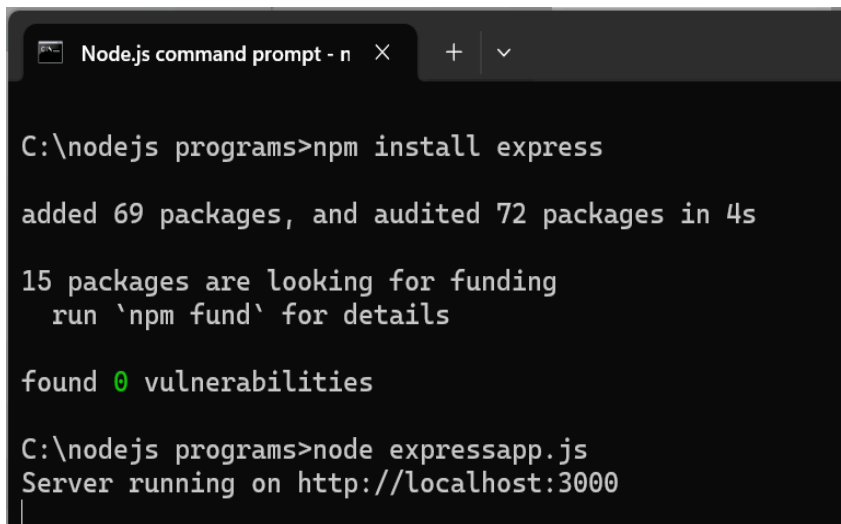
Create a server:

```
const express = require('express');
const app = express();
app.get('/', (req, res) => {
  res.send('Hello, World!This is express');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Run the application:

```
node app.js
```

Output:



```
Node.js command prompt - n  X  +  v

C:\nodejs programs>npm install express

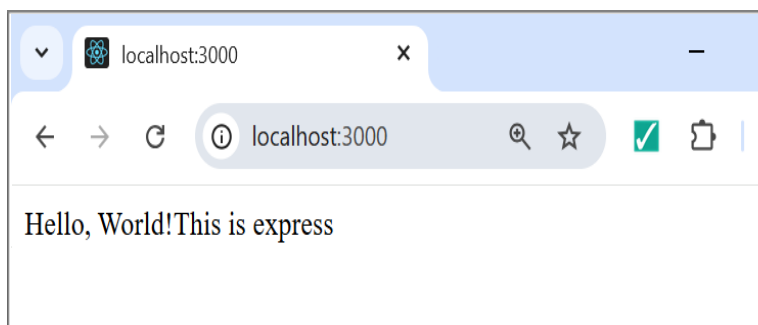
added 69 packages, and audited 72 packages in 4s

15 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

C:\nodejs programs>node expressapp.js
Server running on http://localhost:3000
```

In web browser:



By using `npm install`, you can easily integrate third-party libraries into your Node.js projects and manage dependencies efficiently. It's a core part of Node.js development!

IV. Understanding package.json and package-lock.json

In Node.js, package.json and package-lock.json are essential files that play a crucial role in managing dependencies, scripts, and metadata of a Node.js project. Here's a detailed explanation of each file and their purpose:

- **package.json**

package.json is a JSON file that contains metadata about your Node.js project. It serves as a blueprint for the project, defining its dependencies, versioning, scripts, and other configurations. It is the primary way to describe your project and its requirements.

Key Features of package.json

- **Metadata:** Provides details about your project such as its name, version, description, author, and license.
- **Dependencies:** Lists the libraries/packages your project relies on to function properly.
- **Scripts:** Defines custom commands that can be executed using npm run.
- **Versioning:** Helps manage the version of the project and its dependencies.
- **Configuration:** Includes additional fields for custom behavior or tools like linters, test runners, etc.

Structure of package.json

Here's an example of a typical package.json file:

```
{
  "name": "my-node-project",
  "version": "1.0.0",
  "description": "A sample Node.js project",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Your Name",
  "license": "MIT",
  "dependencies": {
    "express": "^4.18.2",
    "lodash": "^4.17.21"
  },
  "devDependencies": {
    "nodemon": "^2.0.22"
  }
}
```

Important Fields in package.json

- **name:** The name of your project (must be unique for publishing to npm).
- **version:** The version of your project, following semantic versioning (MAJOR.MINOR.PATCH).
- **dependencies:** Lists runtime dependencies (packages needed to run your application).
- **Example:** "express": "^4.18.2"
- **devDependencies:** Lists development-only dependencies (tools used during development but not in production).
- **Example:** "nodemon": "^2.0.22"
- **scripts:** Defines custom scripts for tasks like starting the server, running tests, building, etc. Example: "start": "node index.js"
- **main:** Specifies the entry point of your application (e.g., index.js).
- **license:** Specifies the license under which the project is distributed.

- description: A brief explanation of what the project does.

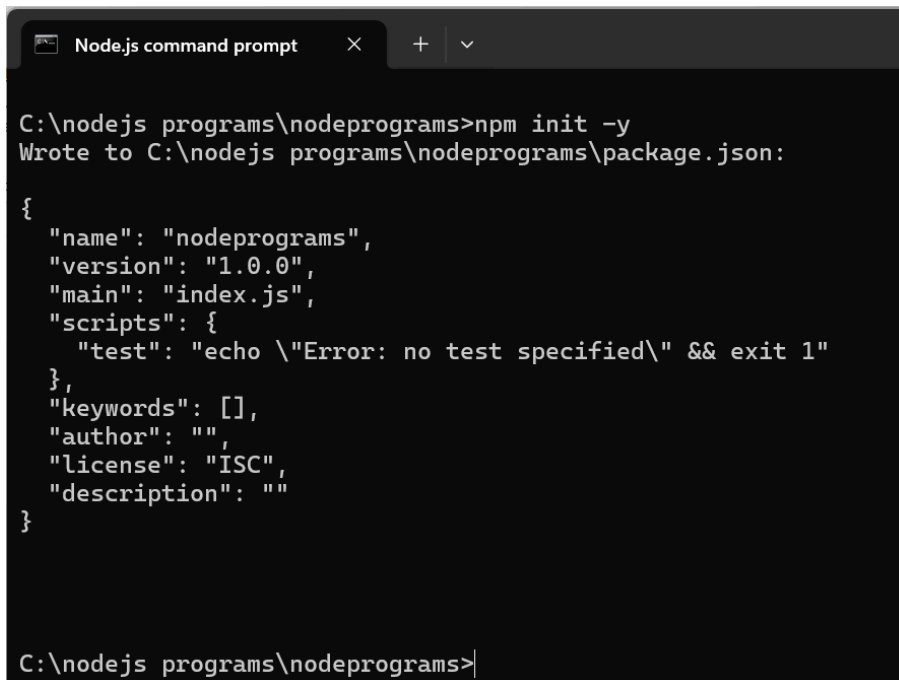
How to Create a package.json?

Run the command:

```
npm init
```

This will prompt you for details to generate the file.

Use **npm init -y** to auto-generate a default package.json with all fields pre-filled.



```

Node.js command prompt
C:\nodejs programs\nodeprograms>npm init -y
Wrote to C:\nodejs programs\nodeprograms\package.json:

{
  "name": "nodeprograms",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}

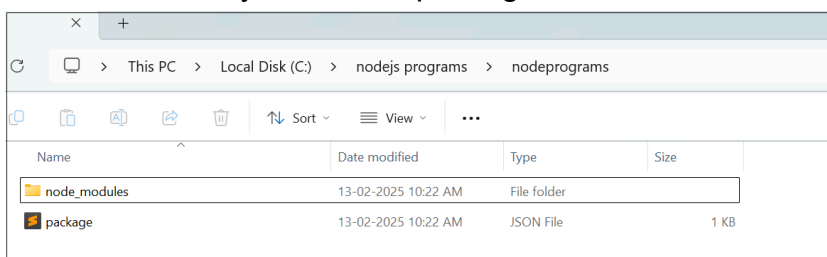
C:\nodejs programs\nodeprograms>

```

• package-lock.json

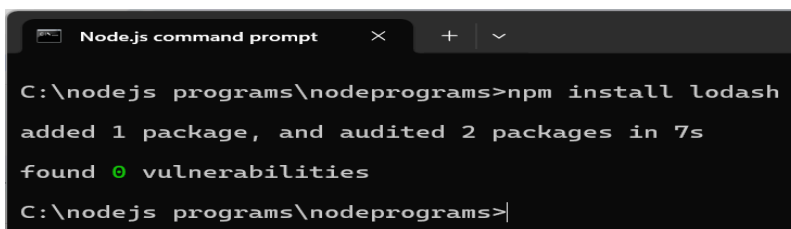
package-lock.json is automatically generated whenever you install or update dependencies using npm. It ensures consistent dependency versions across all environments where the project is used, providing a detailed snapshot of the dependency tree.

Before installation of any module or package:



Name	Date modified	Type	Size
node_modules	13-02-2025 10:22 AM	File folder	
package	13-02-2025 10:22 AM	JSON File	1 KB

Installing any module(e.g, lodash module)

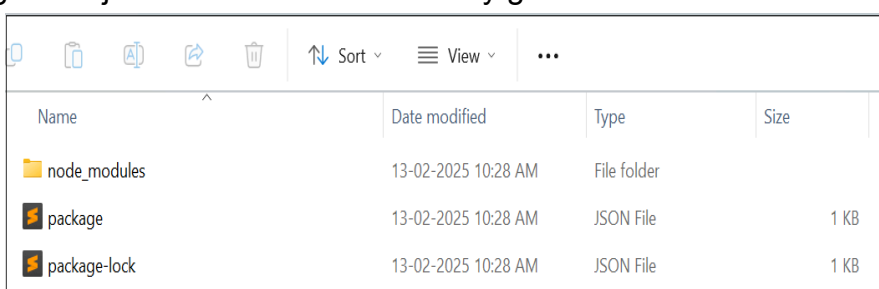


```

Node.js command prompt
C:\nodejs programs\nodeprograms>npm install lodash
added 1 package, and audited 2 packages in 7s
found 0 vulnerabilities
C:\nodejs programs\nodeprograms>

```

package-lock.json has been automatically generated.



Name	Date modified	Type	Size
node_modules	13-02-2025 10:28 AM	File folder	
package	13-02-2025 10:28 AM	JSON File	1 KB
package-lock	13-02-2025 10:28 AM	JSON File	1 KB

Key Features of package-lock.json

Dependency Locking: Locks the exact versions of all installed packages, including nested dependencies, to avoid breaking changes.

Reproducible Builds: Ensures that all environments (development, staging, production) have the same dependency tree.

Improved Performance: Speeds up the installation process by skipping version resolution for already locked packages.

Security: Helps identify and fix vulnerabilities by providing a precise list of all packages and their versions.

Structure of package-lock.json

Here's a simplified example of a package-lock.json file :

```
{
  "name": "my-node-project",
  "version": "1.0.0",
  "lockfileVersion": 2,
  "dependencies": {
    "express": {
      "version": "4.18.2",
      "resolved": "https://registry.npmjs.org/express/-/express-4.18.2.tgz",
      "integrity": "sha512-somehashvalue",
      "requires": {
        "accepts": "^1.3.8",
        "body-parser": "^1.20.1"
      }
    },
    "lodash": {
      "version": "4.17.21",
      "resolved": "https://registry.npmjs.org/lodash/-/lodash-4.17.21.tgz",
      "integrity": "sha512-anotherhashvalue"
    }
  }
}
```

Important Fields in package-lock.json

- **name:** The name of the project (from package.json).
- **version:** The version of the project (from package.json).
- **lockfileVersion:** Specifies the version of the lockfile format.
- **dependencies:** Contains all direct and nested dependencies with their exact versions, URLs, and integrity hashes.

Differences Between package.json and package-lock.json

Aspect	package.json	package-lock.json
Purpose	Describes project metadata and dependencies.	Locks exact versions of all dependencies.
Editable by User	Yes, you can manually edit it.	No, it's auto-generated by npm.
Dependency Versions	Specifies version ranges (e.g., ^4.18.2).	Specifies exact versions (e.g., 4.18.2).
Scope	Top-level dependencies only.	Includes all dependencies (nested too).

Generation	Created during npm init or manually.	Auto-generated during dependency installation or update.
-------------------	--------------------------------------	--

Example: Installing a Package and Its Impact on Both Files

Install the express package:

```
npm install express
```

Impact on package.json:

A new entry will be added under "dependencies":

```
"dependencies": {
  "express": "^4.18.2"
}
```

Impact on package-lock.json:

- Detailed information about the express package, including its exact version and its dependencies, will be added.

Best Practices

- Always commit both package.json and package-lock.json to version control (e.g., Git) for reproducible builds.
- Avoid manually editing package-lock.json; let npm manage it.
- Run npm install without arguments to ensure dependencies match the package-lock.json.

Conclusion

- package.json is the project's blueprint that specifies the necessary dependencies and metadata.
- package-lock.json ensures consistency in dependency installation by locking the exact versions.

These two files work together to make dependency management in Node.js robust, efficient, and reliable.

V. Managing dependencies (devDependencies vs. dependencies)

Managing dependencies effectively is one of the core responsibilities when working with Node.js projects. npm allows developers to categorize dependencies into two main types: dependencies and devDependencies. Let's dive into what they are, how they differ, and when to use each.

• Dependencies

These are the packages/modules required for your application to run in production. They include libraries or frameworks your application relies on during execution.

Example: If you are building a web server with Express, express will be a dependency.

How to install dependencies?

You can install a dependency by running:

```
npm install <package-name>
```

Example:

```
npm install express
```

Impact on package.json

When you install a dependency, it is added under the dependencies section in package.json:

```
"dependencies": {
  "express": "^4.18.2"
}
```

• DevDependencies

These are the packages/modules required only during the development phase of your project. They are not needed in the production environment.

Example: Testing libraries like Jest, or tools like Babel for transpilation.

How to install devDependencies?

You can install a development dependency by running:

```
npm install <package-name> --save-dev
```

OR (short form):

```
npm i <package-name> -D
```

Example:

```
npm install nodemon --save-dev
```

Impact on package.json

When you install a devDependency, it is added under the devDependencies section in package.json:

```
"devDependencies": {  
  "nodemon": "^2.0.22"  
}
```

Key Differences Between Dependencies and devDependencies

Aspect	dependencies	devDependencies
Purpose	Required for the app to run in production.	Required only during development.
Usage	Includes runtime libraries or frameworks.	Includes testing, build, and dev tools.
Examples	express, lodash, mongoose.	jest, nodemon, eslint, webpack.
Installed in Production?	Yes, by default when running npm install.	No, unless explicitly installed.

• Managing Dependencies Based on Environment

- **Default Installation** : By default, running `npm install <package>` will add the package to dependencies.
- **Production Installation** : To install only dependencies (ignoring devDependencies), use the following command:

```
npm install --production
```

This is commonly used when deploying applications to production environments.

- **Install All Dependencies** : Running `npm install` without any flags will install both dependencies and devDependencies.

• Example Scenario

- Project Context

You're building a Node.js web app with the following requirements:

- Use Express to create a web server.
- Use Nodemon to auto-restart the server during development.
- Use Jest for testing.

- Installing Dependencies

```
npm install express # Adds express to dependencies
```

```
npm install nodemon --save-dev # Adds nodemon to devDependencies
```

```
npm install jest --save-dev # Adds jest to devDependencies
```

- Resulting package.json

```
{
```

```

"name": "my-node-project",
"version": "1.0.0",
"description": "Example Node.js project",
"main": "index.js",
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js",
  "test": "jest"
},
"dependencies": {
  "express": "^4.18.2"
},
"devDependencies": {
  "nodemon": "^2.0.22",
  "jest": "^29.6.2"
}
}

```

• Best Practices for Managing Dependencies

- Differentiate Dependencies:
 - Use dependencies for libraries your app needs to run.
 - Use devDependencies for tools that help during development (e.g., testing, linting).
- Version Control:
 - Always commit both package.json and package-lock.json to ensure dependency consistency across environments.
- Use Semantic Versioning:
 - npm uses semantic versioning for dependency versions (e.g., ^, ~).
 - "^4.18.2": Allows updates for minor/patch versions (e.g., 4.19.0 but not 5.0.0).
 - "~4.18.2": Allows updates for patch versions only (e.g., 4.18.3 but not 4.19.0).
- Keep devDependencies Out of Production:
 - Use npm install --production or NODE_ENV=production to skip devDependencies during deployment.
- Audit Dependencies:
 - Use npm audit to check for security vulnerabilities in your dependencies.
- Regularly Update Dependencies:
 - Use npm outdated to list outdated packages and npm update to update them.

npm audit

• Common Issues and Solutions

- **Issue:** Installing a package under the wrong category.
Solution: Move the package to the correct section:


```
npm install <package-name> --save-dev
```
- **Issue:** Dependency version conflicts between environments.
Solution: Use package-lock.json to ensure consistent versions.
- **Issue:** Too many devDependencies in production.
Solution: Use npm install --production during deployment.

Managing dependencies and devDependencies effectively ensures your Node.js application runs smoothly in both development and production. By categorizing packages correctly and adhering to best practices, you can streamline development, avoid unnecessary installations in production, and maintain a clean, well-documented project.

Exercise:

Use GPT and explore dependency auditing and security in npm.

Put the below problem statement in the message box and see what ChatGPT says.

“How does npm help in auditing package dependencies for vulnerabilities? Explain the use of npm audit, how to interpret its output, and steps to fix security issues in a project.”

ChatGPT ▾

Memory Full ⓘ

⬆️ Share

How does npm help in auditing package dependencies for vulnerabilities? Explain the use of npm audit, how to interpret its output, and steps to fix security issues in a project

How npm Helps in Auditing Package Dependencies for Vulnerabilities

`npm audit` is a powerful feature provided by Node Package Manager (npm) to help developers identify and address security vulnerabilities in their project dependencies. It scans the `package-lock.json` or `package.json` file for known vulnerabilities in the installed packages and provides a detailed report.

Using `npm audit`

- Running the Audit:**

Use the following command to perform a security audit:

```
bash
```

⌂ Copy ↻ Edit

Message ChatGPT

+

🌐 Search

💡 Reason

🔊

ChatGPT can make mistakes. Check important info.

After the 1.5 hr ILT, the student has to do a 30 minute live lab. Please refer to the Session 3 Lab doc in the LMS.